Lab Ten – Point Estimation

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use "enter" to make your code readable, so it doesn't run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you're submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don't forget that you can copy-and-paste code when you're doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good `ggplot` practice AND will help make sure you don't make a mistake when computing the probability.

1 Lab 8 Notes

1.1 Altham's Multiplicative Binomial Distribution

The PMF for Altham's Multiplicative Binomial Distribution is

$$f_X(x) = \frac{1}{c}\binom{n}{x}p^x(1-p)^{n-x}\theta^{x(n-x)}I(x \in \{0, 1, ..., n\})$$

where

$$c = \sum_{i=0}^{n}\binom{n}{i}p^i(1-p)^{n-i}\theta^{i(n-i)}.$$

Just so we start on the same page, I have included my `R` function for computing probability using this distribution.

```r
daltbinom <- function(x, size, prob, theta){
  normalizing.constant <- 0
  for(j in 0:size){
    normalizing.constant <- normalizing.constant +
      choose(size, j) * prob^j * (1-prob)^(size-j) * theta^(j*(size-j))
  }
  # Probability Mass at x=x
  (1/normalizing.constant) *
    choose(size, x) * prob^x * (1-prob)^(size-x) * theta^(x*(size-x)) *
      (x>=0)*(x<=size)
}
```

2 Altham's Multiplicative Beta Binomial Distribution

The PMF for Altham's Multiplicative Beta Binomial Distribution is

$$f_X(x) = \left[\int_0^1 \left(\frac{1}{c}\binom{n}{x}p^x(1-p)^{n-x}\theta^{x(n-x)}\right)\left(\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)}p^{\alpha-1}(1-p)^{\beta-1}\right)dp\right] I(x \in \{0, 1, ..., n\})$$

Just so we start on the same page, I have included my `R` function for computing probability using this distribution.

```r
daltbbinom.single <- function(x, size, alpha, beta, theta){
    integrand <- function(p) {
    sapply(p, # integrate will pass in a vector, so we use sapply to apply the following
           function(p_val){
             # f(x|p) * f(p|alpha, beta)
             daltbinom(x, size, p_val, theta) * dbeta(p_val, alpha, beta)
           }
    )
  }
  # Probability Mass at x=x
  # integrate(integrand, lower = 0, upper = 1)$value * (x>=0)*(x<=size)
  # I shrink the bounds below to avoid log(0)=-infinty
  # I use try() to catch when there is an error and avoid crashing
  res <- try(integrate(integrand, lower = 1e-7, upper = 1 - 1e-7)$value *
                (x>=0)*(x<=size),
             silent = TRUE)
  # If there is an error, return a near-zero probability
  if(inherits(res, "try-error")) return(0.1^6)

  res
}
# Write a function that works on a vector
daltbbinom <- function(x, size, alpha, beta, theta){
  sapply(X=x, FUN=daltbbinom.single, size=size, alpha=alpha, beta=beta, theta=theta)
}
```

3 Question 1

In Lab 8, we use Altham's Multiplicative Binomial Distribution. Your job was to interpret the impact of θ in that equation. In this lab, we will model the number of legs won in a best of eleven (first to six) match.

To help us think about the interpretation, something that was challenging before, let's try something a little more manageable.

3.1 Part a

Suppose a player has a 50% chance of winning a specific leg, that is let $p = 0.50$. Let's look at the probability they win ($x = 6$) legs in the match. Compute this probability over a sequence from $\theta = 0.5$ to $\theta = 1.5$ and plot it using a line where the x-axis is θ and the y-axis is $P(X = 6)$.

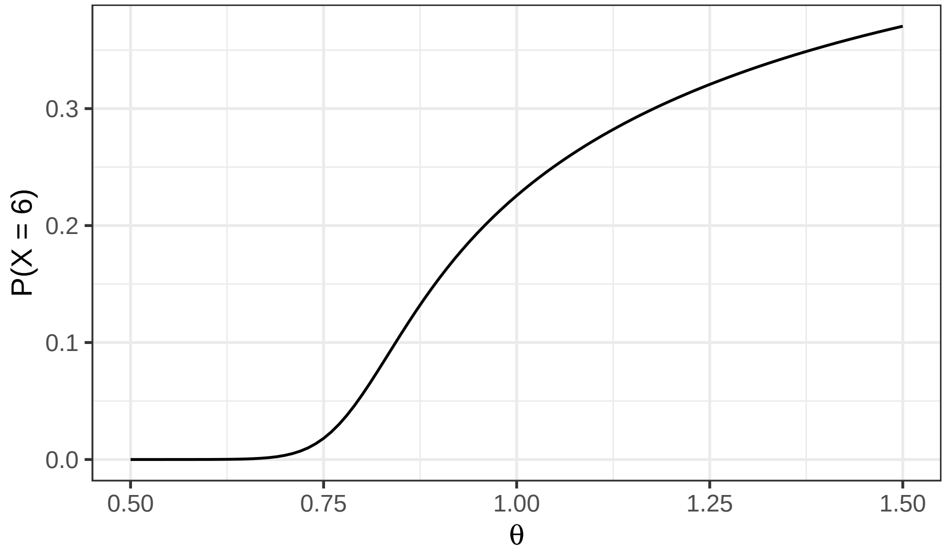
Solution

```r
p = 0.50
x=6
n=11 #define parameters

ggdat = tibble(theta = seq(0.5, 1.5, by = 0.01)) |>
  mutate(prob = daltbinom(x = x, size = n, prob = p, theta = theta)
  )
```

```r
ggplot(ggdat, aes(x = theta, y = prob)) +
  geom_line() +
  labs(x = expression(theta),
    y = "P(X = 6)"
  ) +
  theme_bw()
```

Figure 1: Probability of Winning 6 Legs in a Match as Theta Changes



3.2 Part b

Now, plot Altham's Multiplicative Binomial Distribution under the same parameters with $\theta = 0.50$, $\theta = 1$, and $\theta = 1.5$. Use `ncol=1` in `facet_wrap()` to plot the PMFs in a single column for comparing.

Solution

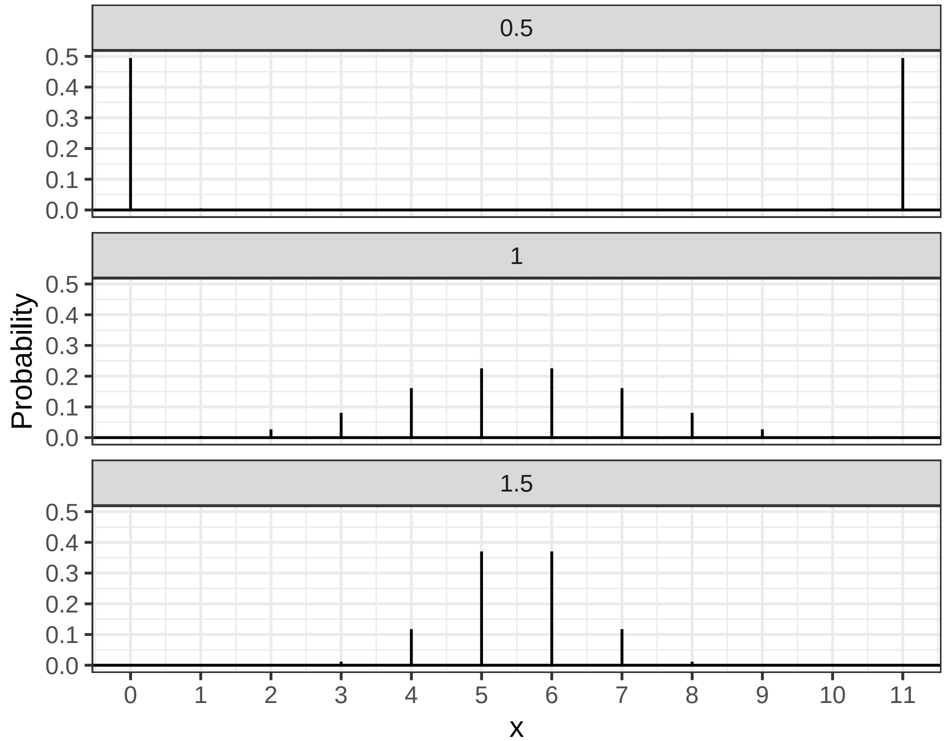
```
p = 0.50
n=11 #define parameters

ggdat = tibble(
  x = rep(0:n, times = 3),
  theta = rep(c(0.5, 1.0, 1.5), each = n + 1)) |>
  mutate(
    PMF = daltbinom(x = x, size = n, prob = p, theta = theta) #define PMF
  )
```

```
PMF.plot = ggplot(ggdat, #plot PMF
  aes(x = x, ymin = 0, ymax = PMF)) +
  geom_linerange() +
  geom_hline(yintercept = 0) +
  facet_wrap(~ theta, ncol = 1) + #facet wrap theta
  scale_x_continuous("x", breaks = 0:n) +
  ylab("Probability") +
  theme_bw()

PMF.plot
```

Figure 2: Altham's Multiplicative Binomial PMF



3.3 Part c

What is your best assessment of what θ is doing in this setting. Consider when a 6-0 blowout is most likely, and when a 5-6 nail-biter is most likely. Note here we should treat the match as a fixed $n = 11$ as we did for $n = 3$ in Lab 8.
Note: In lab 8, we saw that small θ meant wins came in bunches (clumped together), so there were more 2-0 wins in a best of three series.

Solution The parameter θ captures dependence between legs by influencing whether outcomes tend to cluster or alternate. When $\theta < 1$, results are more likely to occur in streaks, producing more extreme outcomes, while $\theta > 1$ leads to more alternation and more balanced match results. However, the model does not explicitly account for the order in which legs occur. Therefore sequences with the same total number of wins are treated identically. As a result, theta captures overall clustering behavior but doesn't model the exact order of outcomes within a match.

4 Question 2

4.1 Part a

Altham's Multiplicative Binomial Distribution does not typically simplify to the clean $E(X) = np$ seen in the Binomial distribution, unless $\theta = 1$. Write out the expression that would need to be solved to find $E(X^k)$, and then write a function that computes it given the proposed parameter values p (`prob`) and θ (`theta`).

Solution

```
E.altbinom.k = function(k, size, prob, theta){ #create function
  x.vals = 0:size #vector of x values
  sum((x.vals^k) * daltbinom(x.vals, size, prob, theta))
}
```

4.2 Part b

Describe how you can use your answer to part a to find Method of Moments estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution You can create a function with (par, data, size) and define the parameters p and theta as parameters 1 and 2. then you set m1 equal to the mean of the given data, m2 equal to the mean of the given data^2, and so on. Then, for E1 and E2, you set these equal to E.altbinom.k (the function from part a) with k = 1,2... Using what you wrote for m1, m2 and E1, E2 do c(E1-m1, E2-m2) which should result in (0, 0). Then use nleqslv, putting in fn, x, data, size to output the Method of Moments estimator based on the specific data. Method of Moments is an approximation, that gives back the maximum likelihood of observing a certain outcome.

4.3 Part c

Write a function that computes the log-likelihood for Altham's Multiplicative Binomial Distribution, given data (`x`) and proposed parameter values p (`prob`) and θ (`theta`).

Solution

```
ll.altham = function(par, data, size, mle=FALSE){
  p = ifelse(mle, 1/(1 + exp(-par[1])), par[1])
  theta = ifelse(mle, exp(par[2]), par[2]) # exp gives the exponential to ensure p and theta are positive

  ll = sum(log(daltbinom(x = data, size = size, prob = p, theta = theta))) #compute log likelihood

  ifelse(mle, -ll, ll)
}
```

4.4 Part d

Describe how you can use your answer to part c to find Maximum Likelihood estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution We can use the log-likelihood function from part c to find Maximum Likelihood Estimates for p and theta by finding the parameter values that maximize the likelihood of the observed data. This can be done later using optim() to search for the values of p and theta that make the observed outcomes most probable. To ensure valid parameters, we can transform p to stay between 0 and 1 and theta to stay positive during optimization (in the ifelse statment in part c), then transform back to report the MLEs. A good starting point for optimization is the Method of Moments estimate from Part b.

5 Lab 10

Peter "Snakebite" Wright is a two-time Professional Darts Corporation World Champion (2020, 2022) and a former World No. 1. He is one of the sport's most decorated players, winning nearly 50 PDC titles including the World Matchplay, UK Open, and multiple European Championships.

In 2024, he made The Premier League – a league involving the eight best players that runs every Thursday night from February to May. In this season, he won two of eighteen matches, securing only four points. The spread in legs won was -43, meaning he lost 43 more legs than he won. This performance was rather quite bad. He was soundly trounced in almost all his match ups.

The 2024 standings are below:

Rank	Player
1	Luke Littler
2	Luke Humphries
3	Michael van Gerwen
4	Michael Smith
5	Nathan Aspinall

Rank	Player
6	Rob Cross
7	Gerwyn Price
8	Peter Wright

5.1 Summarize the 2024 Data

In `pdc2024.csv`, I have provided the data for the 2024 Premier League season. There are a few data cleaning steps to consider.

- Remove any matches with no `Score` (e.g., a bye or walkover)
- Nights 1 through 16 are best of 11 (first to 6), but the playoffs are extended. For consistency, keep nights 1 through 16 only.
- Make two columns `WinnerLegs` and `LoserLegs` that keep track of the number of legs each player has won

Summarize the data. What do the data tell us about the breakdown of the results?

Solution

```
dat.pdc = read.csv("pdc2024.csv")

library(dplyr)
library(tidyr)
  dat.pdc.cleaned = dat.pdc |>
    filter(!is.na(Score)) |>
    filter(!Score %in% c("bye", "walkover")) |>
    filter(Night %in% 1:16) |>
    separate(Score, into = c("WinnerLegs", "LoserLegs")) |>
    mutate(WinnerLegs = as.numeric(WinnerLegs), #make numeric
           LoserLegs = as.numeric(LoserLegs)) |>
    filter(!is.na(WinnerLegs), !is.na(LoserLegs))
```

Figure 3: Distribution of Match Outcomes by Losing Legs

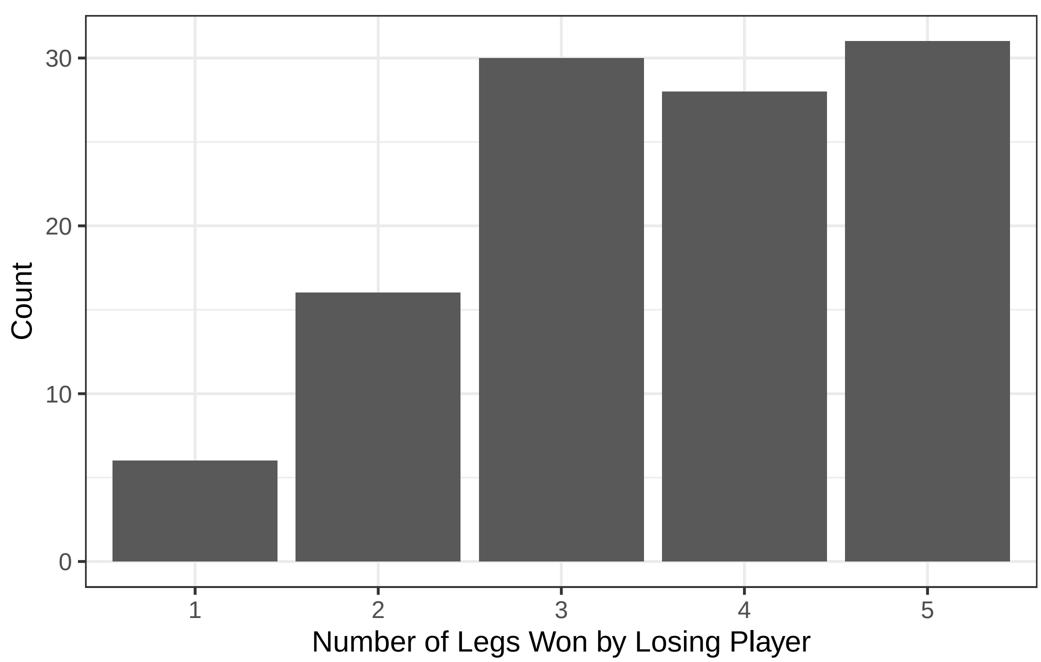


Table 2: Summary of Premier League Scores

mean.winner	mean.loser	sd.winner	sd.loser	mean.diff	sd.diff
6	3.559	0	1.196	2.441	1.196

First I cleaned the data by removing matches without valid scores, taking out byes and walkovers, and restricting the dataset to Nights 1 through 16. The score variable was then separated into WinnerLegs and LoserLegs, which were converted to numeric values.

The distribution of match outcomes shows that losing players most frequently won 4 or 5 legs, indicating that many matches were competitive. Extreme outcomes such as 6–0 or 6–1 were relatively uncommon.

Summary statistics further support this pattern. The average number of losing legs is approximately 3.56, while the average winning total is close to 6, as expected under the match format. The average margin of victory is small (2.44), indicating that most matches are decided by a few legs rather than being one-sided.

Overall, both the distribution and summary statistics suggest that matches in the 2024 season were generally competitive, with relatively few dominant performances

5.2 Altham's Multiplicative Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maximium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLE for the player using this data. Ensure to report p and θ for each player in a nicely formatted table.
  Note: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space. For example, here, you might consider `p <- 1/(1+exp(-par[1]))` and `theta <- exp(par[2])`.
- Make comments about the MLEs in the context of the standings.

Solution

```
library(dplyr)
winners = dat.pdc.cleaned |>
  select(Player = Winner, LegsWon = WinnerLegs)

losers = dat.pdc.cleaned |>
  select(Player = Loser, LegsWon = LoserLegs)

player.data = bind_rows(winners, losers) #combine winner and loser rows

ll.altham = function(par, x, n = 11) {
  p = 1 / (1 + exp(-par[1]))
  theta = exp(par[2])          # transform parameters

  ll = sum(log(daltbinom(x, size = n, prob = p, theta = theta)))

  return(-ll)  # negative for minimization
}
```

```
fit.altham.player = function(df) {
  x = df$LegsWon

  #find MLE
  fit = optim(
    par = c(0, 0),
```

```r
    fn = ll.altham,
    x = x
  )

  tibble(
    p.hat = 1 / (1 + exp(-fit$par[1])),
    theta.hat = exp(fit$par[2])
  )
}
```

Table 3: Maximum Likelihood Estimates for Altham's Model by Player

Player	p.hat	theta.hat
Gerwyn Price	0.383	1.020
Luke Humphries	0.461	1.205
Luke Littler	0.469	1.316
Michael Smith	0.426	1.047
Michael van Gerwen	0.431	1.023
Nathan Aspinall	0.403	1.186
Peter Wright	0.317	0.989
Rob Cross	0.395	1.014

The estimated values of p and θ vary across players, reflecting differences in both ability and match dynamics. The parameter p represents the probability of winning an individual leg. Players with higher estimated values of p tend to perform better in the standings, indicating a relationship between individual leg success and overall match outcomes.

The parameter θ captures dependence between legs. Values of $\theta < 1$ indicate clustering of results, where players are more likely to win or lose multiple legs in succession, leading to more extreme outcomes. In contrast, large values of θ (1, 1.5) correspond to more alternating results and balanced match outcomes.

This interpretation is supported by the values in Table 4, where players with smaller θ tend to exhibit more variability in match outcomes, while those with larger θ show more consistency around close scorelines. Overall, these results suggest that both skill (through p) and dependence between legs (through theta) play important roles in determining match results.

5.3 Altham's Multiplicative Beta Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maxmium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLEs for the player using this data. Ensure to report α, β, and θ for each player in a nicely formatted table.
  Note 1: Due to the computational complexity of `integrate()`, you should compute the logged probability for `0:6` once and add them according to the data in each player's vector. **Note 2**: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space.
- Plot the underlying Beta(α, β) distribution for each player.
  Note: Add the mean and variance of the Beta to the table of MLEs. Recall

$$E(X) = \frac{\alpha}{\alpha + \beta}$$

$$sd(X) = \sqrt{\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}}$$

- Make comments about the MLEs in the context of the standings.
  Note: The players alternate who throws first in a leg has the mathematical "head start" to reach a finish before their opponent can take another turn. Players alternate who throws first with each leg.

Solution

```
ll.altham.bb = function(par, x, n = 11) {

  alpha = exp(par[1])
  beta  = exp(par[2])
  theta = exp(par[3]) #transform parameters

  k.vals = 0:6
  probs = sapply(k.vals, function(k) { #probabilities for each k
    daltbbinom(k, size = n, alpha = alpha, beta = beta, theta = theta)
  })

  ll = sum(log(probs[x + 1]))  # match probabilities to observed data

  return(-ll) # negative for minimization
}
```

```
fit.altham.bb.player = function(df) {
  x = df$LegsWon

  #Compute MLE
  fit = optim(
    par = c(0, 0, 0),
    fn = ll.altham.bb,
    x = x
  )

  alpha = exp(fit$par[1])
  beta  = exp(fit$par[2])
  theta = exp(fit$par[3])

  tibble(
    alpha.hat = alpha,
    beta.hat  = beta,
    theta.hat = theta,
    mean = alpha / (alpha + beta),
    sd = sqrt(alpha * beta / ((alpha + beta)^2 * (alpha + beta + 1)))
  )
}
```

Table 4: MLEs for Altham Beta-Binomial Model by Player

Player	alpha.hat	beta.hat	theta.hat	mean	sd
Gerwyn Price	0.264	0.917	3.175	0.224	0.282
Luke Humphries	0.331	0.490	6.391	0.403	0.363
Luke Littler	0.283	0.384	10.797	0.425	0.383
Michael Smith	0.534	1.068	2.245	0.334	0.292
Michael van Gerwen	0.219	0.454	5.001	0.325	0.362
Nathan Aspinall	2.468	4.338	1.492	0.363	0.172
Peter Wright	1.786	7.289	1.214	0.197	0.125
Rob Cross	0.268	0.813	3.202	0.248	0.299

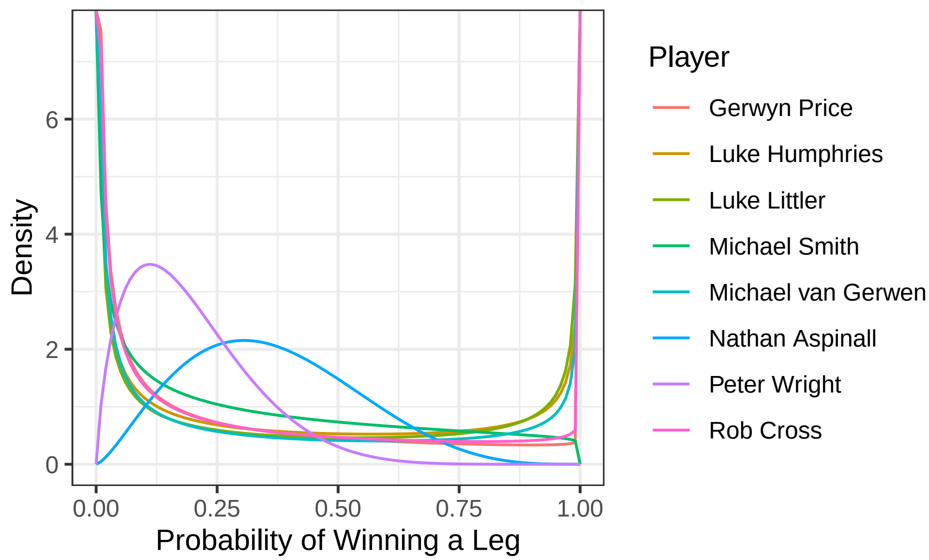
```r
library(dplyr)
library(tidyr)
library(ggplot2)

x.vals = seq(0, 1, length.out = 100)

# Expand the data frame so each player has a row for each x
player.density = player.results.bb |>
  select(Player, alpha.hat, beta.hat) |>
  crossing(x.val = x.vals) |>
  rowwise() |>
  mutate(density = dbeta(x.val, alpha.hat, beta.hat)) |>
  ungroup()

ggplot(player.density, aes(x = x.val, y = density, color = Player)) +
  geom_line() +
  labs(
    x = "Probability of Winning a Leg",
    y = "Density"
  ) +
  theme_bw()
```

Figure 4: Estimated Multiplicative Beta-Binomial Distributions by Player



The multiplicative Beta-Binomial model extends the Binomial framework by allowing the probability of winning a leg to vary across matches. The estimated parameters α and β define a Beta distribution, where the mean represents the average probability of winning a leg and the standard deviation refers to variability in performance. Players with higher estimated means tend to perform better overall, while nonzero standard deviations indicate fluctuations in performance across matches and opponents.

The parameter θ continues to capture dependence between legs, with values less than 1 indicating clustering and values near 1 suggesting approximate independence.

This model provides a better fit to the observed data because it accounts for both differences in player ability and variability in performance. This is supported by Table 5, where players exhibit both differing means and nonzero variability, patterns that are not captured by a standard Binomial model with a fixed probability.

Overall, the Beta-Binomial model offers a more realistic representation of match dynamics by incorporating both heterogeneity in player performance and dependence between legs.

5.4 Executive Summary

Summarize the work you've done here to provide a brief (200-300 word) data-driven summary of the 2024 Premier League season using your work in Lab 10. Your summary should be professional, clear, and all claims should be corroborated with statistical support.

Solution

The 2024 Premier League Darts season was very competitive, with most matches decided by relatively small margins. After cleaning the dataset to include only standard-format matches (Nights 1–16), the distribution of outcomes shows that losing players most frequently won 4 or 5 legs, while extreme outcomes such as 6–0 or 6–1 wins were uncommon. Summary statistics further supported this pattern, with an average of approximately 3.56 losing legs per match and a relatively small average margin of victory, indicating that most matches were very close.

To better understand player performance and match dynamics, Altham's multiplicative Binomial model was used to estimate each player's probability of winning a leg (p) along with the incorporation of the dependence parameter (θ). Players with higher estimated values of p generally performed better in standings, confirming that leg-level success translates into greater match success. The parameter θ captures dependence between legs, where values less than 1 indicate clustering of outcomes (streaks or extremes), while values near or above 1 suggest more alternating results. This demonstrates that match outcomes are not fully independent from leg to leg.

Extending the analysis to the multiplicative Beta-Binomial model allows the probability of winning a leg to vary across matches. The estimated means and standard deviations of the Beta distribution show both differences in player ability and variability in performance, patterns that are not captured by a standard Binomial model.

Overall, the Beta-Binomial model provides the best representation of match dynamics for this case by accounting for both dependence between legs and variation in player performance, offering a more complete statistical description of the 2024 season.